CHAPTER 1

INTRODUCTION

Railway transportation is one of the most widely used modes of transport in India, serving millions of passengers every day. With such a large user base, passengers frequently require information related to train schedules, PNR status, crowd conditions, and onboard facilities. Traditionally, this information is obtained through official railway portals or enquiry counters, which may become congested or difficult to access during peak hours.

The rapid advancement of web technologies has made it possible to design user-friendly systems that present information in a structured and accessible manner. In an academic environment, however, building real-time systems that depend on live databases or APIs is often impractical due to security, access limitations, and technical complexity. As a result, simulation-based systems using mock data are widely adopted for learning and demonstration purposes.

The Track Engine project is developed as a web-based railway enquiry simulation system. It demonstrates how different railway information modules can be integrated into a single interface using mock data. The project focuses on system design, user interaction, and frontend logic rather than real-time deployment.

1.1. Problem Introduction

1.1.1 Motivation and Objectives

The motivation behind developing Track Engine arises from the need to understand how large-scale information systems are structured and presented to users.

The key objectives of this project are:

- To design a centralized railway enquiry system using web technologies
- To simulate PNR status, train status, crowd level, and pantry information
- To demonstrate modular design and separation of concerns

- To provide a simple and intuitive user interface
- To develop a system suitable for academic evaluation and demonstration

This project enables students to gain hands-on experience in frontend development and system simulation.

1.1.2 Scope of the Project

The scope of the Track Engine project is limited to the design and development of a web- based railway enquiry simulation system using mock data. The project focuses on simulating essential railway services such as PNR status, train running status, crowd information, and pantry details through a single user interface. The system is intended for academic and learning purposes and does not involve real-time data, backend servers, or external APIs. The project scope includes frontend development, user input validation, modular system design, and data simulation. Features such as real-time updates, database integration, payment systems, and authentication are beyond the scope of this Mini Project and may be considered for future enhancement.

1.2 Related Previous Work

Several railway enquiry systems and web applications have been developed to provide real-time information to passengers, including official railway portals and mobile applications. These systems primarily rely on centralized databases and live data sources to display train schedules, ticket status, and service details. However, implementing real-time systems in an academic environment is complex due to data access restrictions. Therefore, simulation-based projects using mock data have been widely adopted for educational purposes.

1.3 Organization of the Report

This report is organized into four chapters and two appendices.

- Chapter 1: introduces the project, its objectives, scope, and motivation.
- Chapter 2: presents the literature review, discussing existing railway enquiry systems and simulation-based approaches used in academic projects.

- Chapter 3**:** describes the system design and implementation details of the Track Engine project, including architecture and module descriptions.
- Chapter 4: concludes the report by summarizing the outcomes and achievements of the project.

CHAPTER 2

LITERATURE SURVEY

Railway enquiry systems play an important role in providing passengers with information related to train schedules, ticket status, and onboard services. With the growth of web technologies, several systems have been developed to simplify access to such information through online platforms. This literature survey studies existing railway enquiry systems, simulation-based academic projects, and frontend web technologies used to model information systems. The focus of the survey is to understand how similar problems have been addressed, the technologies used, and the limitations faced by real-time systems. The study also highlights the relevance of mock-data-based simulation systems for academic learning and Mini Project development.

2.1 Existing Railway Enquiry Systems

Existing railway enquiry systems such as official railway websites and mobile applications provide real-time information related to PNR status, train schedules, delays, and seat availability. These systems are generally connected to centralized databases and live servers to fetch updated information. They offer accurate data but require high system availability, secure access, and continuous maintenance. Due to heavy user traffic, such systems may experience delays or performance issues during peak hours. While these systems are effective for public use, they are complex to implement in academic projects because they require real-time data access and backend infrastructure**.**

2.2 Web-Based Information Systems

Web-based information systems are widely used to present structured data to users through graphical interfaces. These systems typically use a combination of HTML for content structure, CSS for presentation, and JavaScript for interactivity. Such systems allow users to input data and receive immediate responses. The Track Engine project follows this approach by implementing a web-based interface to simulate railway enquiry services .

2.3 Simulation Systems Using Mock Data

Simulation-based systems use predefined datasets known as mock data to represent real-world information. These systems do not rely on live databases or external APIs, making them suitable for controlled testing and demonstrations. Mock data allows predictable output and simplifies debugging. Track Engine uses mock data to simulate PNR status, train running information, crowd levels, pass validity and pantry services.

2.4 Frontend Technologies for System Development

Frontend technologies such as HTML, CSS, and JavaScript are commonly used to develop interactive user interfaces. HTML provides the basic structure of web pages, CSS enhances visual presentation, and JavaScript adds dynamic behaviour. These technologies are lightweight, widely supported, and ideal for Mini Projects. The Track Engine project effectively utilizes these technologies to create a responsive and user-friendly simulation system.

2.5 Limitations of Existing Approaches

Although real-time railway enquiry systems provide accurate information, they require continuous internet connectivity and backend support. Simulation systems using mock data, on the other hand, do not reflect real-time changes and may lack dynamic updates. Future improvements can include database integration or API connectivity to overcome these limitations.

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

3.1 Product Perspective

The Track Engine system is a standalone, web-based railway enquiry simulation application developed for academic purposes. It is not a component of any larger operational railway system and does not interact with real-time databases or official railway servers. The product simulates commonly used railway enquiry services using predefined mock data to demonstrate system functionality and user interaction.

Track Engine is conceptually similar to existing railway enquiry portals but differs in implementation, as it avoids live data sources and backend integration. The application interacts only with end users through a graphical user interface and does not require integration with external systems.

3.1.1 System Interfaces

The Track Engine system does not interface with external systems such as databases, APIs, or third-party services. All data used by the system is stored locally in the form of mock datasets. Therefore, there are no external system interface requirements for this project.

3.1.2 User Interfaces

The system provides a Graphical User Interface (GUI) accessible through a web browser. The interface allows users to enter details such as PNR number or train number and view corresponding simulated results. The interface is designed to be simple, intuitive, and suitable for users with basic computer knowledge.The interface layout is optimized for clarity, with clearly labeled input fields, buttons, and result sections. Since the system targets general student users, no special accessibility requirements are mandated. However, readable fonts, proper contrast, and consistent navigation are maintained to improve usability.

3.1.3 Hardware Interfaces

The Track Engine system does not directly interact with any specific hardware devices. It runs on standard computing devices such as desktops, laptops, or mobile devices through a web browser.

3.1.4 Software Interfaces

The Track Engine system requires the following software components for execution:

Name	Mnemonic	Specification	Version	Source
Web Browser	Browser	HTML5/CSS3/JS	Latest	Google Chrome / Mozilla Firefox
Code Editor	IDE	VS Code	Latest	Microsoft
Operating System	OS	Windows/Linux	Any	User System

The system does not interface with external application software or databases. All interactions occur within the browser using JavaScript logic and mock data.

3.1.5 Communication Interfaces

The system does not implement any custom communication protocols. If hosted online, it uses standard HTTP/HTTPS protocols provided by the browser and hosting platform. No explicit communication interface handling is required within the application.

3.1.6 Memory Constraints

There are no strict memory constraints imposed on the system. The application is lightweight and designed to operate within the memory limits of standard personal computers and mobile devices. Mock data usage ensures minimal memory consumption.

3.1.7 Operations

The system operates in a single-user interactive mode. Users manually input data and receive simulated results instantly. No unattended or background processing operations are required. Since no persistent storage is used, backup and recovery operations are not applicable.

3.1.8 Site Adaptation Requirements

No special site adaptation is required for installation or operation of the Track Engine system. The application can be executed on any device with a modern web browser. No additional hardware, software installation, or environment configuration is necessary.

3.2 Product Functions

The Track Engine system performs the following major functions:

- Accepts user input for railway-related queries
- Displays simulated PNR status information
- Displays simulated train running status
- Shows predefined crowd level information
- Displays pantry menu details
- Validates user input and handles incorrect entries
- Provides clear and organized output for each module

These functions are logically related and presented through a unified interface.

3.3 User Characteristics

Users are expected to have basic computer literacy and familiarity with web browsers. No prior technical expertise in programming is required to operate the system. These characteristics influence the design of a simple, intuitive, and minimal user interface.

3.4 Constraints

The system is subject to the following constraints:

- No real-time data access
- No backend database integration
- Browser-dependent execution
- Security limited to frontend validation

3.5 Assumptions and Dependencies

It is assumed that users have access to a modern web browser and a basic computing device. The system depends on the availability of JavaScript execution within the browser. Any change in browser compatibility may affect system behavior.

3.6 Apportioning of Requirements

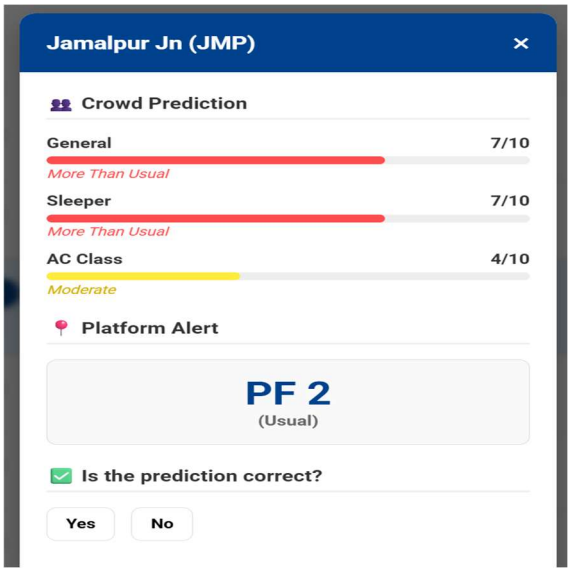
Features such as real-time API integration, database connectivity, user authentication, and mobile application development are identified as future enhancements. The current version focuses only on simulation using mock data.

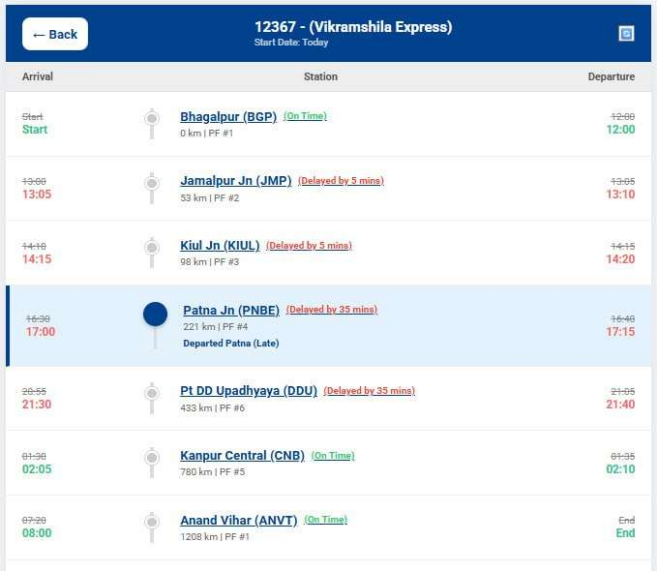
3.7 Use Case

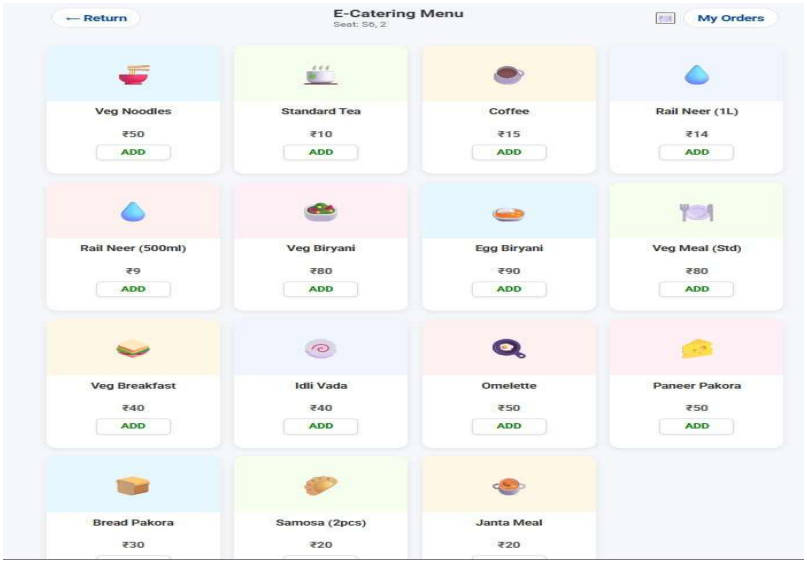
3.7.1 Use Case Model

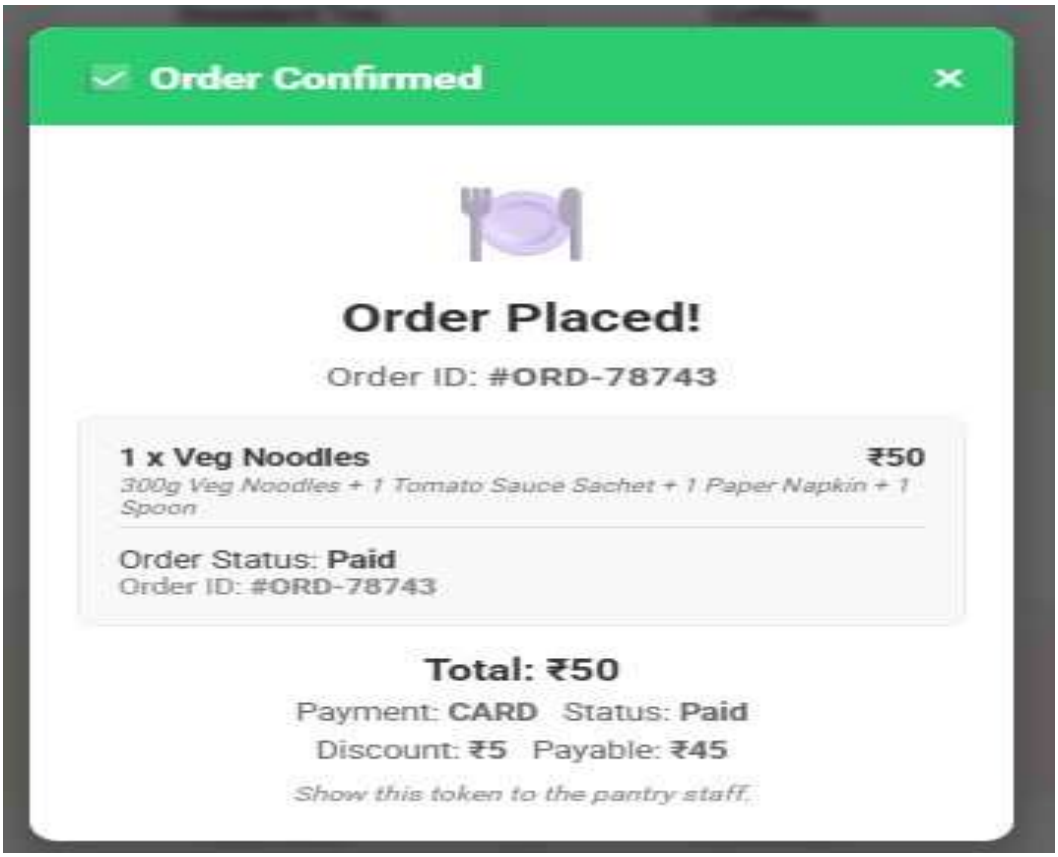
The primary actor of the system is the user. The user interacts with the Track Engine system to access simulated railway information. The system responds to user requests by displaying appropriate data.

3.7.2 Use Case Diagram





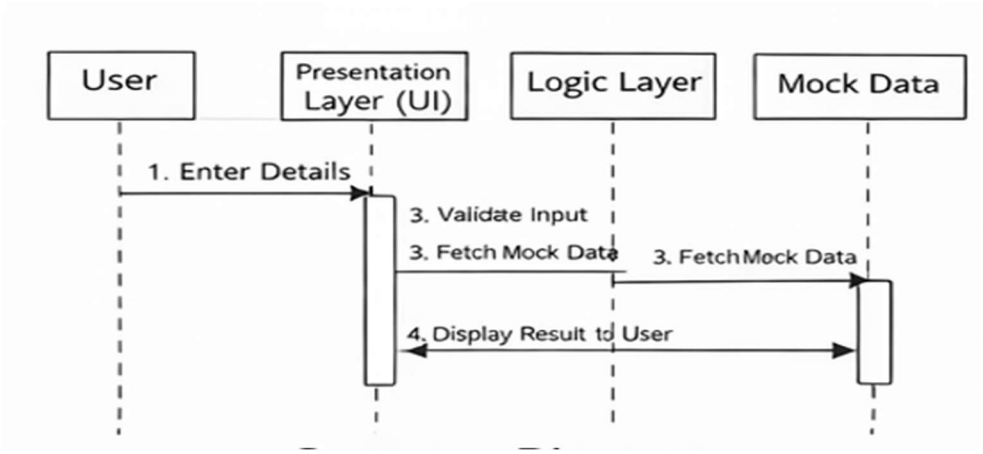

← Return
E-Catering Menu
Seat: S6, 2
My Orders
Veg Noodles
₹50
ADD
Standard Tea
₹10
ADD
Coffee
₹15
ADD
Rail Neer (1L)
₹14
ADD
Rail Neer (500ml)
₹9
ADD
Veg Biryani
₹80
ADD
Egg Biryani
₹90
ADD
Veg Meal (Std)
₹80
ADD
Veg Breakfast
₹40
ADD
Idli Vada
₹40
ADD
Omelette
₹50
ADD
Paneer Pakora
₹50
ADD
Bread Pakora
₹30
Samosa (2pcs)
₹20
Janta Meal
₹20


Order Confirmed
Order Placed!
Order ID: #ORD-78743
1 x Veg Noodles
₹50
300g Veg Noodles + 1 Tomato Sauce Sachet + 1 Paper Napkin + 1 Spoon
Order Status: Paid
Order ID: #ORD-78743
Total: ₹50
Payment: CARD Status: Paid
Discount: ₹5 Payable: ₹45
Show this token to the pantry staff.

3.7.3**Use case scenarios**

Use Case Element	Description
Use Case Number	UC-01
Application	Track Engine
Use Case Name	View PNR Status
Description	User checks simulated PNR status
Primary Actor	User
Precondition	Application is loaded
Trigger	User enters PNR number
Basic Flow	System displays PNR data
Alternate Flow	Invalid PNR message displayed

3.8 Sequence Diagram



CHAPTER 4

SYSTEM DESIGN

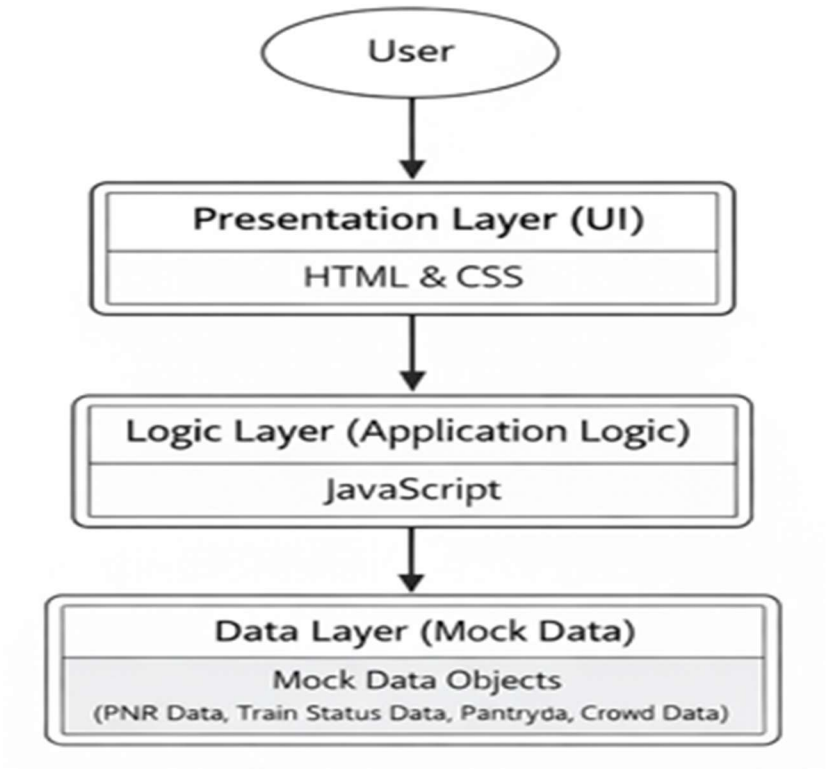
This chapter describes the overall system design of the Track Engine web-based application. The system design explains how different components of the system interact with each other and how data flows through the system. Various diagrams such as architecture diagrams, class diagrams, data flow diagrams, activity diagrams, ER diagrams, and database schema diagrams are used to represent the system visually. Each diagram is referred to in the respective section for better understand.

4.1 Architecture Diagrams

The Track Engine system follows a 3-Tier architecture model, which separates the system into presentation, logic, and data layers. This architecture improves modularity, maintainability, and scalability of the application.

The presentation layer consists of the user interface developed using HTML and CSS, which allows users to interact with the system. The application logic layer is implemented using JavaScript, which processes user inputs, validates data, and retrieves appropriate data. The data layer consists of predefined mock datasets stored locally within the application.

3-Tier architecture of the Track Engine system.



4.1. 3-Tier architecture of the Track Engine Diagram

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data moves through the Track Engine system. In the Level 0 DFD, the user provides input such as PNR number or train number. The system processes the input and retrieves corresponding data from the data repository. The processed information is then displayed to the user.

The DFD helps in understanding the logical flow of data and system boundaries without showing implementation details.

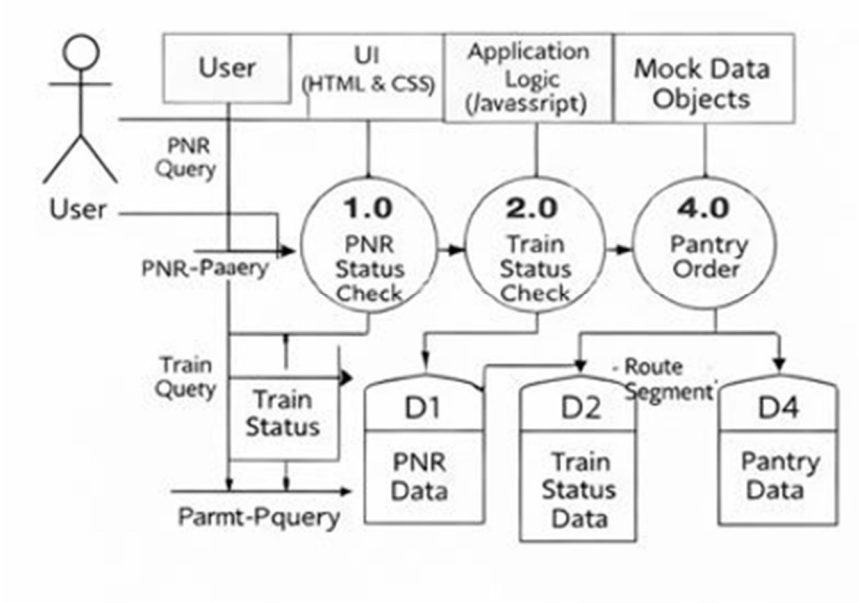


Figure 4.2: Data Flow Diagram of Track Engine

4.3 Activity Diagram

The activity diagram represents the flow of activities performed by the user while interacting with the system. For Track Engine, the activity diagram demonstrates steps such as loading the application, entering input data, validating input, fetching mock data, and displaying results.The activity diagram for user interaction in Track Engine

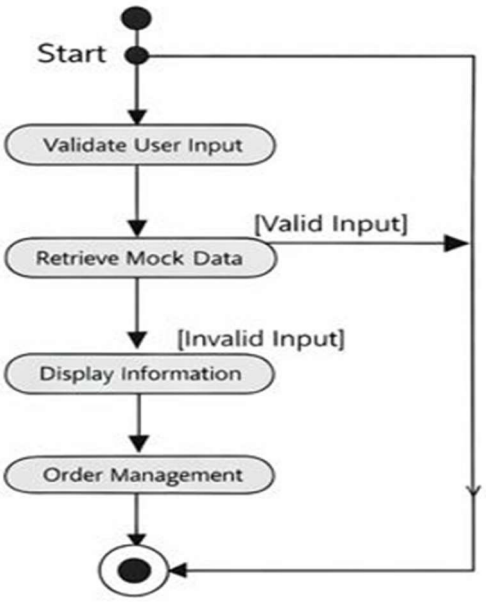


Figure 4.3: Activity Diagram

4.4 ER Diagrams

The Entity Relationship (ER) diagram represents the conceptual data model of the system. The ER diagram helps visualize how data entities such as User, PNR, Train, Crowd, and Pantry are logically related.

The ER diagram defines entities, attributes, and relationships, providing a clear understanding of the system's data organization.

The ER diagram for the Track Engine system

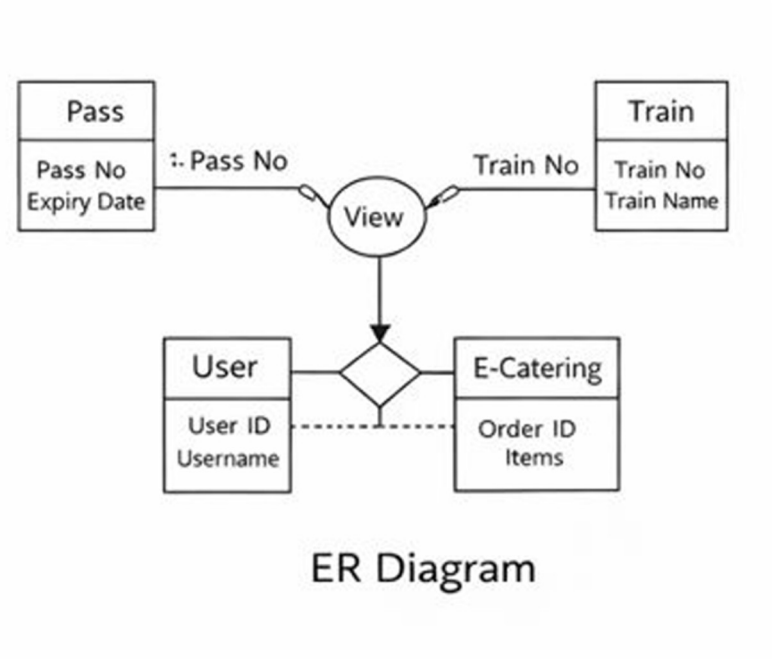


Figure 4.4: ER Diagram of Track Engine

CHAPTER 5

IMPLEMENTATION AND RESULTS

This chapter describes the software and hardware requirements of the Track Engine system, implementation details, testing performed, and the results obtained. The system is implemented as a web-based application using mock data to simulate railway enquiry services. The results validate that the system functions correctly and meets the objectives of the project.

5.1. Software and Hardware Requirements

5.1.1Software Requirements

- Operating System: Windows / Linux / macOS
- Web Browser: Google Chrome or Mozilla Firefox
- Development Tools: Visual Studio Code
- Technologies Used: HTML, CSS, JavaScript

5.1.2Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: 200 MB free disk space
- Display: Standard monitor or laptop screen

The above requirements are sufficient to develop and execute the Track Engine application smoothly.

5.2 Assumptions and Dependencies

The Track Engine system assumes that the user has access to a modern web browser with JavaScript enabled. The system depends on browser compatibility for proper execution of frontend scripts. Since mock data is used, the application does not depend on internet connectivity after loading the web pages.

5.3 Constraints

The following constraints apply to the Track Engine project:

- The system does not use real-time railway data
- No backend database is implemented
- Functionality is limited to simulation using mock data
- Security is limited to basic input validation

5.4 Implementation Details

- The Track Engine system is implemented as a client-side web application.
- HTML is used to structure the user interface, CSS is used for styling and layout, and JavaScript is used to implement logic, validation, and data handling.
- Mock data is stored in JavaScript objects and arrays and is accessed based on user input.
- Each functional module operates independently, ensuring modularity and ease of maintenance.

5.4.1 Snapshots of Interfaces

The following screenshots represent the user interface of the Track Engine system:

- Home Page displaying navigation options
- PNR Status simulation page
- Train Status simulation page
- MyOrder Information page
- Pantry Information page

Each interface is designed to be user-friendly and clearly displays the output generated from mock data.

5.4.2 Test Cases

Table 5.1: Test Cases for Track Engine

Test Case ID	Test Description	Input	Expected Output	Result
TC-01	Check PNR Status	Valid PNR	PNR details displayed	Pass
TC-02	Invalid PNR Input	Invalid PNR	Error message shown	Pass
TC-03	Train Status Check	Valid Train No	Train status displayed	Pass
TC-04	Crowd Info Display	Train selected	Crowd level shown	Pass
TC-05	Pantry Menu View	Pantry option	Menu displayed	Pass

The test cases confirm that all modules function correctly for valid and invalid inputs.

5.4.3 Results

The results obtained from the execution of the Track Engine project show that the system successfully simulates railway enquiry services. The application responds quickly to user inputs and displays accurate data. All test cases passed successfully, indicating correct system behaviour. The modular structure ensures smooth interaction between components. The results demonstrate that the project objectives have been achieved.

CHAPTER 6

CONCLUSION

This chapter summarizes the overall work carried out in the Track Engine project, evaluates system performance, compares the proposed system with existing solutions, and discusses possible future enhancements. The chapter is intentionally kept concise and clearly written to ensure easy understanding by both technical and non-technical readers.

6.1 Performance Evaluation

- The performance of the Track Engine system was evaluated based on responsiveness, accuracy of simulated results, and usability.
- Since the application is entirely frontend-based and uses mock data, it demonstrates fast response time with minimal processing delay.
- User inputs are validated efficiently, and results are displayed instantly without noticeable lag.
- The modular structure of the system ensures smooth execution of individual components without affecting overall performance.
- The system performed reliably during testing, and all functional modules operated as expected within the defined scope of the project.

6.2 Comparison with Existing State-of-the-Art Technologies

- Existing railway enquiry systems provide real-time information by integrating with live databases and official APIs. While such systems offer accurate and up-to-date data, they involve complex backend infrastructure, high maintenance costs, and security considerations.

- In contrast, Track Engine is a simulation-based system designed for academic use. It does not rely on real-time data or external services, making it simpler, lightweight, and easier to understand. Although it does not match the functionality of state-of-the-art real-time systems, it effectively demonstrates core concepts of system design, user interaction, and data handling.

6.3 Future Directions

The work carried out in the Track Engine project suggests several opportunities for future enhancement.

- The system can be extended by integrating real-time railway APIs to provide live PNR and train status information.
- A backend database can be added to store user data and enquiry history.
- From a practical perspective, the project demonstrates how simulation-based systems can be used effectively in academic environments to understand real-world applications.
- Future developers can build upon this foundation to create scalable and production-ready railway information systems.

APPENDIX

The Track Engine system uses predefined demo (mock) data to simulate real-world railway enquiry services. This data is stored locally within the application and is used for testing and demonstration purposes.

The demo data includes:

- PNR Data: Simulated passenger reservation details
- Train Status Data: Predefined train running information
- Crowd Information Data: Crowd levels such as Low, Medium, and High
- Pantry Data: Sample food items and pricing information

HTML CODE:-

```
<div id="orders-view" class="hidden">
    <div class="status-header alt">
      <button onclick="returnFromOrders()" class="back-pill">← Return</button>
      <div class="train-title">
        <h2 class="page-title">My Orders</h2>
      </div>
    </div>
    <div class="search-card" style="max-width:800px; margin:20px auto;">
      <h3>Your Recent Orders</h3>
      <div id="orders-list" style="margin-top:12px;"></div>
    </div>
  </div>
```

CSS CODE:-

```
#pantry-menu-view { max-width: 800px; margin: 0 auto; min-height: 100vh; background-color: #f5f7fa; padding-bottom: 90px; }

.pantry-container { padding: 15px; display: grid; grid-template-columns: repeat(auto-fill, minmax(160px, 1fr)); gap: 15px; }
```

.food-card { *background*: white; *border-radius*: 12px; *overflow*: hidden; *box-shadow*: 0 2px 8px rgba(0,0,0,0.08); *text-align*: center; *padding-bottom*: 12px; }

.food-img-box { *height*: 100px; *background*: #eef2f5; *display*: flex; *align-items*: center; *justify-content*: center; *font-size*: 30px; *color*: #888; }

.food-img-box i { *font-size*: 34px; }

JS CODE:-

function viewOrder(*orderId*) {

const user = getCurrentUser(); if(!user) { openGlobalLogin(); return; }

const ord = user.orders && user.orders.find(*o* => *o*.id === *orderId*);

if (!ord) { alert('Order not found.'); return; }

// populate receipt modal with this order

document.getElementById('receipt-id').innerText = ord.id;

const receiptList = document.getElementById('receipt-summary'); receiptList.innerHTML = '';

for (*const* [id, q] of Object.entries(ord.items || {})) {

const it = pantryMenu.find(*x* => *x*.id == id);

if (it) {

receiptList.innerHTML += `<div class="receipt-row"><div class="receipt-item-name"><span>${q} x ${it.name}</span><span>₹${it.price * q}</span></div><div class="receipt-item-desc">${it.desc}</div></div>`;

}

}

document.getElementById('receipt-total-pay').innerText = `₹${ord.total}`;

document.getElementById('receipt-payment-method').innerText = (ord.method||'--').toUpperCase();

document.getElementById('receipt-payment-status').innerText = ord.status;

document.getElementById('receipt-discount-amount').innerText = `₹${ord.discount}`;

document.getElementById('receipt-payable-amount').innerText = `₹${ord.payable}`;

```
    // show cancel button when appropriate
    const cancelRow = document.getElementById('receipt-cancel-row');
    if (ord.method === 'cod' && ord.status === 'Pending') { cancelRow.style.display = 'block';
window._lastOrder = { id: ord.id, status: ord.status }; }
    else { cancelRow.style.display = 'none'; }


    document.getElementById('receipt-modal').classList.remove('hidden');
}
```

References

- W3Schools, “HTML, CSS and JavaScript Tutorials,” **https://www.w3schools.com**
- Mozilla Developer Network (MDN), “JavaScript Documentation,” **https://developer.mozilla.org**
- Indian Railways, “Passenger Information System – Conceptual Overview,” Available: **https://indianrailways.gov.in**